From Primer to Lean: A retrospective on formalising the anharmonic Laplace asymptotic

Daniel Murfet

April 2026

Abstract

This document is a retrospective on a single morning of formalisation work in Lean 4 + Mathlib, in which the SLT Susceptibility Primer's anharmonic covariance formula

$$\mathrm{Cov}_t[x^2, x] \;=\; -\frac{2\alpha}{\lambda^3 t^2} + o(t^{-2}) \qquad (t \to \infty)$$

was formalised end-to-end, producing 3279 lines of Lean across ten files, 60+ proved theorems, and zero `sorry`, `axiom`, or `native_decide` occurrences. The collaboration involved Claude (Anthropic, Opus 4.7) as the primary working partner with three targeted consultations to GPT-5.5 Pro at strategic decision points. The formalisation borrowed structure and tooling from Geoffrey Irving's `aks` repository. End-to-end the session took 5h 44m of wall-clock time (2h 30m of API time) and $198.53 of model usage. The aim of this retrospective is to record honestly what proportion of the work was Mathlib lookup, what was new, where progress stalled, and how the multi-model workflow contributed.

Contents

1 Setting the stage

In a working note dated 27 April 2026 ("Formalisation First Steps"), DM sketched a path forward for formalisation at Timaeus, building on conversations with Geoffrey Irving and Edmund Lau:

> It's natural to go after big theorems. Geoffrey's `aks` repo is super cool. It would be tempting for us to follow suit and go after one of the main theorems of SLT. However that seems to have two problems: it's unclear if the timing is right, given model capability increases and the gaps in Mathlib; and even if we were to formalise say the free energy formula, it's not immediately clear what that buys us in terms of forward momentum across our research.
>
> [...] If I reflect on where the bottleneck is on using Claude/GPT to go faster on theoretical research, it is the verification step. [...] So in this world I just tell Claude to go "do the lower-order cases of this kind of asymptotic expansion with Wick's theorem" and it goes and writes Section 4 of the primer above, verifies Lemma 20 and then just presents me with the LaTeX and some Lean files. I then read the Lean file for Lemma 20 and just have to verify that the statement is indeed what I think and what the LaTeX says, which takes me 20 minutes.

This retrospective is the first concrete experiment along that path. The "Lemma 20" referred to is the one-dimensional anharmonic case of the Laplace expansion in Section 4 of the SLT Susceptibility Primer (Elliott–Murfet, 2026), and "some Lean files" is the `timaeus-research/laplace` repository whose construction we record here.

The motivation for picking this particular target was:

- The mathematics is genuinely useful but routine. It is the kind of "boring calculation that I am 100% sure I can do if I just sit down for an afternoon" but for which DM does not have a free afternoon. Formalising it is pure win: the calculation is fun but DM is not learning anything new by doing it himself.
- It is well within the joint scope of Opus 4.7 + GPT-5.5 Pro, but non-trivially so: there are sub-goals (Mill's-ratio tail bounds, Taylor remainders, asymptotic algebra) that exercise different parts of Mathlib.

- It can be inspected independently: anyone reading the primer can read `cov_anharmonic_asymptotic` in Lean and check, in 20 minutes, that the statement matches.
- Building it forces the development of a small library of pieces (scalar Taylor bounds, 1D Gaussian moments, Mill's-ratio bounds, rescaling identities) that will be reused by subsequent work in Timaeus's research agenda.

2 The thing to be formalised

The headline result of the formalisation is:

> *For the anharmonic potential*
>
> $$L(x) \;=\; \frac{\lambda}{2}x^2 + \frac{\alpha}{6}x^3 + \frac{\gamma}{24}x^4$$
>
> *with $\lambda > 0$, $\gamma > 0$, and the discriminant condition $\alpha^2 < 3\lambda\gamma$,*
>
> $$\mathrm{Cov}_t[x^2, x] \;=\; -\frac{2\alpha}{\lambda^3\, t^2} + o(t^{-2}) \quad \text{as } t \to \infty,$$
>
> *where $\mathrm{Cov}_t[\phi, \psi]$ is the covariance under the Gibbs measure $e^{-tL(x)}\, dx/Z(t)$.*

In Lean (writing `R` for the type of real numbers, which the Lean source spells $\mathbb{R}$):

```
theorem cov_anharmonic_asymptotic
    {lam alpha gamma : R}
    (hlam : 0 < lam) (hgamma : 0 < gamma) (hdisc : alpha ^ 2 < 3 * lam * gamma) :
    Filter.Tendsto
      (fun t : R =>
        t ^ 2 *
          Laplace.gibbsCov (anharmonicPotential lam alpha gamma) t
            (fun x => x ^ 2) (fun x => x))
      Filter.atTop
      (nhds (-2 * alpha / lam ^ 3))
```

Where this lives in the primer. This is equation (4.10) in the primer, and it is the lowest-dimensional concrete payoff of the asymptotic machinery there. The primer derives it via Wick contractions on the rescaled coordinate $u = x\sqrt{\lambda t}$, expanding $e^{-s_t(u)}$ to first order in the cubic and quartic perturbations.

Why the discriminant condition. The primer's calculation is formal: it expands $e^{-tL(x)}$ as a Gaussian times a perturbation series. For the resulting moment integrals to be well-defined and dominated by the quadratic minimum at $x = 0$, L must be coercive (bounded below by a positive multiple of x^2) globally. This is automatic in the multivariate primer setting under generic-position assumptions, but in 1D with a cubic term it requires the discriminant condition $\alpha^2 < 3\lambda\gamma$. Without it, L can have lower minima away from 0, and the Laplace asymptotic is not driven by $x = 0$.

The hypothesis was added at GPT-5.5 Pro's insistence during the strategy consultation; see §5.1.

3 Background from the primer

The primer's Section 4 computes asymptotic expansions of expectations under $p_t(x) \propto e^{-tL(x)}$. The mechanism, in one dimension, is:

1. Rescale $u = x\sqrt{\lambda t}$. Then p_t becomes $\propto e^{-u^2/2 - s_t(u)}\,du$ where

$$s_t(u) \;=\; \frac{\alpha}{6\lambda^{3/2}}\frac{u^3}{\sqrt{t}} + \frac{\gamma}{24\lambda^2}\frac{u^4}{t}$$

   collects the cubic and quartic perturbations.

2. Expand $e^{-s_t} = 1 - s_t + \frac{1}{2}s_t^2 - \cdots$. To get $O(t^{-2})$ accuracy on $\mathrm{Cov}_t[x^2, x]$ it suffices to keep the first-order term and bound the rest.

3. Compute the leading rescaled moments $M_k = \int u^k e^{-u^2/2}\,du = (k-1)!!\sqrt{2\pi}$ for even k, zero for odd k.

4. Assemble: $\langle x\rangle_t, \langle x^2\rangle_t, \langle x^3\rangle_t$ from the $J_n(t) = \int u^n e^{-u^2/2 - s_t(u)}\,du$, the $I_n(t) = \int x^n e^{-tL(x)}\,dx$ via the substitution $(\sqrt{\lambda t})^{n+1} I_n = J_n$, and finally $\mathrm{Cov}_t[x^2, x] = \langle x^3\rangle_t - \langle x^2\rangle_t\langle x\rangle_t$.

5. The leading t^{-2} coefficient is the algebraic identity $-5\alpha/(2\lambda^3) - (1/\lambda)\cdot(-\alpha/(2\lambda^2)) = -2\alpha/\lambda^3$, which is real-valued ring algebra.

This is what gets formalised. The primer treats it informally; the Lean proof has to make every domination, every tail bound, every change of variable, and every algebraic step explicit.

4 Borrowings from `aks`

Geoffrey Irving's `aks` repository provided the model and several concrete artefacts:

- **The discipline.** `aks` formalises the Agrawal–Kayal–Saxena primality theorem with a strict "no `sorry`, no `axiom`, no `native_decide`" rule. We adopted the same rule for `laplace`, including the audit script.
- `scripts/sorries`**.** The audit tool that scans the codebase for `sorry`, `#exit`, `native_decide`, and `axiom` occurrences was lifted nearly verbatim from `aks/scripts/sorries`. It was run before every commit.
- `scripts/lean-search`**.** The Python wrapper around `leansearch.net` for semantic Mathlib search was also lifted from `aks`. It was used heavily during stage 2 (Gaussian moments) and stage 3 (tail bounds).
- `CLAUDE.md` **structure.** The project guidance file follows the `aks` pattern: a short statement of the goal, a list of build commands, conventions (toolchain pinning, no `native_decide`, etc.), a Mathlib API reference that grows as the project goes, and a tactic-gotchas section. The intent — captured by Geoffrey's repo and adopted here — is that an AI assistant who has not seen the project before should be able to pick up the playbook from `CLAUDE.md` alone.
- **Lake configuration template.** The `lakefile.toml` and `lean-toolchain` pinning style (matching Mathlib at the `v4.29.0` tag) was lifted directly from `aks`.

- **Workflow philosophy.** The aks-style discipline of "skeleton correctness > filling sorries": a `sorry` with a correct statement is valuable, while a `sorry` with a wrong statement is actively misleading. We followed this throughout: at any intermediate point the file built, with placeholder `sorry`s that were filled in only once we were confident the statements were right.

These borrowings turned what would have been a multi-day setup (toolchain configuration, build infrastructure, audit tooling, prompt scaffolding) into a 30-minute setup and let the project start working on actual mathematics from the first turn. Quoting DM's earlier note: "By grabbing some of Geoffrey's aks pieces, Claude + Lean is already making some progress in 30min this morning."

5 Stages of the formalisation

The work proceeded in eight rough stages. For each I record the headline theorem(s), the proportion of work that was Mathlib lookup vs. new, and where applicable the points at which DM intervened.

5.1 Stage 0: Strategy consultation

Before writing any Lean, Claude proposed three approaches to the primer's formula:

1. Rescaled Gaussian + perturbation expansion (the primer's own route).
2. Watson–Erdélyi asymptotic expansion of parameter-dependent integrals.
3. Morse normal form via local inverse function theorem.

GPT-5.5 Pro was consulted on which to pick.

> **GPT-5.5 Pro:** Use (1), but don't localise first. With the right hypothesis, you can prove a global first-order perturbative bound and skip most of the ugly tail machinery for this stage. *First: the statement needs a hypothesis fix.* As written, with only $\lambda > 0$, $\gamma > 0$, $\alpha \in \mathbb{R}$, the claim is false in general: if α is large enough, L has lower minima away from 0, so Laplace is not dominated by $x = 0$. A sufficient condition for 0 to be the unique global minimum is $\alpha^2 < 3\lambda\gamma$.

This was the first decisive intervention. The hypothesis fix ($\alpha^2 < 3\lambda\gamma$) made the rest of the proof tractable, and the recommendation to do a global first-order bound (rather than a localisation theorem in the Watson style) avoided multi-week infrastructure work on inverse function theorems and Jacobian expansions. GPT-5.5 Pro's strategy memo is preserved at `gpt_responses/strategy_consultation.md`.

5.2 Stage 1: Gibbs definitions and the scalar Taylor cornerstone

Files: `Laplace/Gibbs.lean` (65 lines), `Laplace/ScalarBound.lean` (179 lines).

The Gibbs definitions (`partitionFunction`, `gibbsExpectation`, `gibbsCov`) were straightforward Mathlib applications; nothing new.

The scalar Taylor cornerstone is

$$\left| e^{-z} - (1 - z) \right| \;\le\; \tfrac{z^2}{2} \cdot \max(1, e^{-z}),$$

proved as `abs_exp_neg_sub_one_add_le`. This is the load-bearing inequality for the entire perturbative bound: every later $O(1/t)$ estimate ultimately reduces to multiplying both sides of this by a Gaussian, integrating, and bounding by Gaussian moments.

Mathlib gap: this exact form was not in Mathlib. The closely related `Real.add_one_le_exp` ($1 + z \le e^z$) is, but it gives only the one-sided bound. The cornerstone splits on the sign of z: above zero, $e^{-z} - 1 + z \le z^2/2$ by Taylor; below zero, $e^{-z} - 1 + z \le (z^2/2)e^{-z}$ by the same Taylor expansion of e^{-z} around $z = 0$ but with the second-derivative bound shifted. Both halves were written from scratch.

5.3 Stage 2: 1D Gaussian moments

File: `Laplace/OneD/GaussianMoments.lean` (231 lines).

The standard moments $M_{2k} = (2k-1)!!\sqrt{2\pi}$ and $M_{2k+1} = 0$.

Mathlib provided:

- `integral_gaussian` (the base case $\int e^{-bx^2}\,dx = \sqrt{\pi/b}$).
- `integral_rpow_mul_exp_neg_mul_rpow` (general Gamma-function form for half-line integrals).
- `Real.Gamma_nat_add_half` ($\Gamma(k + 1/2) = (2k-1)!!\sqrt{\pi}/2^k$).
- `Nat.doubleFactorial` and its recurrences.
- `integral_comp_abs` (folding the real line to $(0, \infty)$ via $|x|$).

What was new: stitching these into the four user-facing forms (`integral_pow_mul_exp_neg_sq_half_Ioi`, `integral_pow_mul_exp_neg_sq_half`, `gaussian_moment_normalized`, and `integral_pow_mul_exp_neg_t_sq_half`) involved a surprising amount of bridge work: converting between x^{2k} (natural-number power) and $x^{2k \cdot 1.0}$ (rpow), folding even powers via $|x|^{2k} = x^{2k}$, and feeding $1/2$ into exponents without having it eaten by an over-eager `simp`. The recurring tactic gotchas from this stage are recorded in `CLAUDE.md` (e.g. *cascading `rw [(1/2) = 2⁻¹]` stomps inside exponents*).

Proportion: mathematically, all the content is Mathlib's. The work was bookkeeping.

5.4 Stage 3: Anharmonic potential and coercivity

File: `Laplace/OneD/Anharmonic.lean` (140 lines).

The headline theorem is `anharmonic_coercive`: under $\alpha^2 < 3\lambda\gamma$, there exists $c > 0$ with $L(x) \ge c\,x^2$ for all x. Proof: complete the square inside $\frac{\lambda}{2} + \frac{\alpha}{6}x + \frac{\gamma}{24}x^2$ and use the discriminant.

Mathlib provided: `discrim` and the basic theory of quadratic forms.

What was new: the application to this specific anharmonic potential, written by Claude. About 90% of this stage was new; only the discriminant lemma is from Mathlib.

5.5 Stage 4: Tail bounds and localisation

Files: `Laplace/OneD/TailBound.lean` (532 lines), `Laplace/OneD/Localisation.lean` (164 lines).

A small Mill's-ratio family: bounds of the form

$$\int_M^\infty x^k e^{-bx^2/2}\,dx \;\le\; (\text{polynomial in } M) \cdot e^{-bM^2/2}$$

for $k = 0, 1, 2$ and rescaled exponents.

Mathlib provided: the basic Gaussian integrals and the fundamental theorem of calculus over half-lines. The tail bounds themselves were not in Mathlib at the form needed.

What was new: the entire 532-line tail-bound development. In hindsight the localisation file was over-engineered for the eventual proof: GPT-5.5 Pro's strategy of doing a *global* first-order

remainder bound (§5.1) made these tail bounds essentially unused. They are kept because they will be useful for future work on multivariate Laplace asymptotics, where global bounds are not available.

This was the first place where Claude noticeably overshot the strict needs of the target. The second was §5.6.

5.6 Stage 5: Harmonic Gibbs (an over-engineered detour)

File: `Laplace/OneD/Harmonic.lean` (165 lines).

Closed-form expectations under the harmonic Gibbs measure e^{-tL_λ} with $L_\lambda(x) = \frac{\lambda}{2}x^2$: $\langle x^{2k}\rangle = \frac{(2k-1)!!}{(\lambda t)^k}$, odd moments zero, and the trivial covariance $\mathrm{Cov}_t[x^2, x] = 0$ in the harmonic case.

Why this was a detour. An early plan (before GPT-5.5 Pro's strategy memo) was to compute the anharmonic case as a perturbation *around* the harmonic case in original coordinates. After the strategy memo, the actual proof works in rescaled coordinates and never uses the harmonic-Gibbs closed forms directly. This file is kept as a sanity check (the rescaled-coordinate moments must agree with the original-coordinate moments at $\alpha = \gamma = 0$) and as a piece of infrastructure for future work.

Mathlib provided: all the moment integrals from stage 2. **What was new:** the lifting to the Gibbs setting, mostly algebraic massaging.

5.7 Stage 6: The rescaling identity and pointwise perturbation bound

File: `Laplace/OneD/Rescaling.lean` (230 lines).

The two analytic cornerstones:

1. `anharmonic_rescaling_identity`: $tL(u/\sqrt{\lambda t}) = u^2/2 + s_t(u)$.
2. `rescaled_max_decay`: $e^{-u^2/2}\cdot\max(1, e^{-s_t(u)}) \le e^{-c_0 u^2}$ under coercivity, with $c_0 = \min(1/2, c/\lambda)$.

Mathlib provided: `Real.exp_le_exp` and basic estimates. **What was new:** the algebraic identity (a one-line calculation written carefully), and the decay bound (which combines the coercivity estimate from stage 3 with the rescaling identity).

5.8 Stage 7: The integral remainder and J_n asymptotics

File: `Laplace/OneD/IntegralRemainder.lean` (1541 lines, about half the project).

This is the analytic core. The chain of theorems is:

- `perturbation_remainder_pointwise` + `perturbation_remainder_combined`: pointwise $|u^n e^{-u^2/2}(e^{-s_t} - (1 - s_t))| \le C/t \cdot |u|^n(u^6 + u^8)e^{-c_0 u^2}$.
- `perturbation_remainder_integral_bound`: integrating the pointwise bound, $|\int \cdots| \le K/t$.
- `linearised_integral_decomposition`: $\int f(u)(1-s_t(u))\,du = M_n - (A/\sqrt{t})M_{n+3} - (B/t)M_{n+4}$.
- `J_n_asymptotic`: the master J_n asymptotic for general n.
- `J_0_asymptotic`, `J_1_asymptotic`, `J_2_asymptotic`, `J_3_asymptotic`: parity-specialised specifications.
- `I_n_J_n_relation`: $(\sqrt{\lambda t})^{n+1} I_n = J_n$ (substitution identity).

- `I_0_asymptotic`, `I_1_asymptotic`, `I_2_asymptotic`, `I_3_asymptotic`: original-coordinate moment asymptotics.
- `cov_coefficient_identity`: $-5\alpha/(2\lambda^3) - (1/\lambda)\cdot(-\alpha/(2\lambda^2)) = -2\alpha/\lambda^3$.
- Four `tendsto_*` theorems converting the asymptotic bounds to `Filter.Tendsto` statements.
- `cov_anharmonic_J_form_asymptotic`: the headline result in the rescaled J_n form.
- `cov_anharmonic_asymptotic`: the headline result in the original `gibbsCov` form.

Mathlib provided: integration linearity (`integral_add`, `integral_sub`, `integral_const_mul`), change of variables (`Measure.integral_comp_mul_right`), the `Filter.Tendsto` algebra and asymptotic notation (`Asymptotics.IsBigO`, `Asymptotics.IsLittleO`).

What was new: essentially all of it. About 1500 lines of Lean specific to this proof.

5.9 The intervention that broke the J-form / gibbsCov bridge

The last step — bridging `cov_anharmonic_J_form_asymptotic` (rescaled-J_n form) to `cov_anharmonic_asymptotic` (original `gibbsCov` form) — got stuck for several hours. The goal, after unfolding definitions and substituting $I_n = J_n/(\sqrt{\lambda}t)^{n+1}$, was a single rational identity:

$$t^2 \cdot \frac{Z\cdot I_3 - I_2\cdot I_1}{Z^2} = \frac{\sqrt{t}\cdot(J_0J_3 - J_2J_1)}{\lambda\sqrt{\lambda}\cdot J_0^2}.$$

This is true. The variables are $\lambda, t, J_0, J_1, J_2, J_3$ and the square roots $\sqrt{\lambda}, \sqrt{t}, \sqrt{\lambda t}$ which appear in the substitution identity. Standard tactics — `ring`, `field_simp; ring`, `linear_combination`, `polyrith`, `nlinarith` with the relevant `Real.sq_sqrt` facts — all failed: `ring` cannot reason about $\sqrt{\cdot}$, and the others did not find the right combination.

After three failed attempts DM noticed:

> **DM:** Seems like you might be struggling a bit, remember you can always share your troubles with 5.5Pro.

GPT-5.5 Pro's response was the third decisive intervention:

> **GPT-5.5 Pro:** Use fresh variables for the square roots, then rewrite `lam` and `t` as squares before `field_simp`. `linear_combination/polyrith` are the wrong tools here; this is a single rational identity, not a linear combination of hypotheses. After `rw [← hsl2, ← hst2]`, there are no sqrt axioms left: everything is in the polynomial variables `sl, st`. Then `field_simp` turns the goal into a genuine polynomial identity, and `ring` closes it.

The recipe: introduce `set sl := Real.sqrt lam`, `set st := Real.sqrt t`; prove $\texttt{sl}^2 = \lambda$, $\texttt{st}^2 = t$; rewrite λ and t as $\texttt{sl}^2, \texttt{st}^2$ in the goal; then `field_simp` alone (no `ring` needed). This worked immediately. The fix is preserved verbatim at the end of `IntegralRemainder.lean` (lines 1527–1539).

GPT-5.5 Pro's bridge memo is preserved at `gpt_responses/bridge_help.md`.

6 Where I got stuck and how 5.5 Pro helped

Three GPT-5.5 Pro consultations bracket the project. Each was a strategic inflection point.

Strategy (§5.1). Picked the right approach (rescaled Gaussian, global first-order bound) before Claude could spend days building Watson–Erdélyi infrastructure. Identified the missing hypothesis ($\alpha^2 < 3\lambda\gamma$) without which the theorem is false. Proposed the exact form of the load-bearing scalar Taylor lemma ($e^{-z} - (1-z)$ bound) that became the cornerstone in stage 1. **Without this consultation, the project might never have finished.**

Progress check. After completing the analytic core, Claude proposed a generic parity framework for J_n that would have added significant bookkeeping. GPT-5.5 Pro's response:

> **GPT-5.5 Pro:** Skip generic step 10/11. Specialise immediately to $n = 0, 1, 2, 3$ and work with the rescaled unnormalized moments $J_n(t)$. This avoids a generic parity framework and most bookkeeping. Avoid `Real.rpow` if possible. For $n = 0, 1, 2, 3$, write scaling in terms of `Real.sqrt` and integer powers.

This redirected the work away from a generic infrastructure and toward four targeted specifications. The advice on avoiding `Real.rpow` turned out to be especially load-bearing: `Real.rpow` interacts badly with `nlinarith` and with the `Real.sqrt` idioms used elsewhere in the project, and the bridge step in §5.9 would have been substantially harder if the moment scaling were stated via `rpow`. The full memo is at `gpt_responses/progress_check.md`.

Bridge help (§5.9). The final-step unblock: the recipe of substituting fresh symbols for the square roots and rewriting λ, t as squares before `field_simp`. Without this advice the project would have had to either (a) keep trying tactics in the dark, (b) abandon the bridge and present only the rescaled-J_n form, or (c) abandon λ-as-a-parameter and work over a fixed $\lambda = 1$ baseline.

Pattern. GPT-5.5 Pro's contributions were uniformly *strategic*: choosing the right approach, identifying the right hypothesis, picking the right tactic. None of the consultations involved GPT writing Lean code. Claude wrote all the Lean. GPT chose the road.

7 What was Mathlib, what was new

A rough accounting, in lines of Lean:

File	Lines	Mathlib	New content
`Gibbs.lean`	65	100%	Definitions only
`ScalarBound.lean`	179	30%	The Taylor cornerstone
`GaussianMoments.lean`	231	80%	Bridge work
`Anharmonic.lean`	140	10%	Coercivity
`TailBound.lean`	532	5%	Mill's-ratio family
`Localisation.lean`	164	5%	Tail localisation
`Harmonic.lean`	165	60%	Closed forms
`Rescaling.lean`	230	5%	Identity + decay
`IntegralRemainder.lean`	1541	10%	Analytic core
Total	3247	∼20%	∼80%

("Mathlib" here is shorthand for "content essentially supplied by Mathlib lemmas, modulo minor packaging". "New" means proofs we wrote. The percentages are eyeball estimates.)

The Mathlib gaps. The largest gaps were:

- No two-sided $|e^{-z} - (1-z)|$ bound. Mathlib has `Real.add_one_le_exp` but not the symmetric version with $\max(1, e^{-z})$.
- No Mill's-ratio family at the strength needed (`TailBound.lean`). Mathlib has `ProbabilityTheory.gaussian` results but not the polynomial-times-Gaussian half-line bounds.
- No global first-order Laplace remainder lemma. The 200-line `perturbation_remainder_integral_bound` is the workhorse and has no Mathlib analogue.
- No off-the-shelf substitution identity $(\sqrt{\lambda t})^{n+1} I_n = J_n$. The change-of-variables result `Measure.integral_co` is in Mathlib, but the specific application here needed careful packaging.

Where Mathlib carried us. `Filter.Tendsto`, `Asymptotics.IsBigO`, the integral linearity lemmas, the basic Gaussian integral, the double-factorial / Gamma function bridge, and the real-analysis tactic infrastructure (`positivity`, `linarith`, `nlinarith`, `ring`, `field_simp`) were all indispensable. About 70% of the project would have been infeasible without these.

8 Where DM intervened

Across the (single morning) session, DM's interventions fell into a small number of categories.

Setting the goal.

> ***DM:*** *I want to try with Lean formalisation of some of the Wick contraction / asymptotic expansions of* $\exp(-tL)$ *in the primer, what do you think?*

> ***DM:*** *Where is that in the primer?*

> ***DM:*** *Ok let's get the exact formula in a Lean theorem please.*

DM steered Claude away from generic asymptotic-expansion infrastructure toward the specific primer formula, and refused to accept a partial result that stopped short of the named covariance.

Sanity-checking the trajectory.

> ***DM:*** *Can you explain how what we formalised actually proves this formula about* $\mathrm{Cov}_t[x^2, x]$*?*

This came after stage 7's I_n asymptotics were complete. Claude wrote out the algebraic manipulation $\langle x^3 \rangle - \langle x^2 \rangle \langle x \rangle$, which surfaced (a) the fact that the Gibbs ratios I_n / I_0 had not yet been proved to admit asymptotics in the right form and (b) the missing coefficient identity. Both were filled in.

Pushing through to the end.

> ***DM:*** *This is great. Feel free to consult with 5.5 Pro about next steps, but I'd like us to continue.*

> ***DM:*** *Continue, let's just go all the way now.*

A recurring pattern was Claude proposing to "stop at a clean intermediate result" (e.g. stopping at the I_n asymptotics; stopping at the rescaled-J_n-form theorem). DM consistently declined these exits and insisted on going all the way to a Lean theorem stated in `gibbsCov` form.

The 5.5 Pro intervention.

> ***DM:*** *Seems like you might be struggling a bit, remember you can always share your troubles with 5.5 Pro.*

This is the prompt that produced the bridge fix in §5.9. DM noticed Claude was thrashing on the same goal across multiple attempts (the signs from `CLAUDE.md`: "oscillating between approaches, growing helper count without progress, repeated restructuring"), and suggested escalating to GPT-5.5 Pro before sinking more time. It is the kind of intervention that an experienced research supervisor learns to make for graduate students: stop, look up, ask for help.

Categories of intervention. Mapping the pattern from the `coevolving_coefficients` retrospective onto this project:

- *Directive* (telling Claude exactly what to write): rare here. The directive moments were almost all setup-related ("borrow Geoffrey's audit script", "pin Mathlib at v4.29.0", "add a `CLAUDE.md`").
- *Exploratory* (open-ended questions): also rare. The most notable was "How much of this is background material that was previously missing in mathlib, versus tools that are strictly speaking 'part of the proof'?", which produced a useful audit.
- *Goal-pinning*: dominant. Most interventions were of the form "yes, but go further" or "yes, but match the primer".
- *Escalation prompts*: the GPT-5.5 Pro consultations were all DM-initiated; Claude did not propose them on its own.

9 Observations on the multi-model workflow

Cost ratio. Three GPT-5.5 Pro consultations vs. 3000 lines of Lean. The asymmetry is striking: a few minutes of GPT time produced the strategic skeleton; many hours of Claude time filled it in. This matches the pattern in the `coevolving_coefficients` retrospective: GPT was a strategic checkpoint, Claude was the working partner.

No code from GPT. Unlike in the grammar-paper collaboration (where GPT-5.4 Pro produced explicit calculations and ladder-algebra formulas), in this project GPT-5.5 Pro produced no Lean code. Its contributions were strategic (which approach), structural (which hypothesis), and tactical (which Lean idiom). This may reflect the different shape of the task: formalisation requires less novel mathematical content and more tactical engineering, and the engineering is best done in continuous interaction with the Lean compiler — which Claude has and GPT does not.

Claude's failure mode: thrashing. Claude's pattern when stuck was to produce more variants of the same approach (try `linear_combination`, then `polyrith`, then `nlinarith`, then `linear_combination` with hand-crafted hypotheses) rather than stepping back and asking what the right *tactic class* for the goal was. The bridge step was eventually solved by recognising that the goal was a rational identity (so the right tool is `field_simp`) modulo the square-root algebra (so the right move is to substitute symbols for the square roots). Neither move requires a flash of mathematical insight; both are standard moves on a flow chart for "prove an algebraic identity in Lean". But Claude did not make them on its own; the prompt to consult 5.5 Pro was needed.

Claude's overshoot: over-engineering. Two stages (`TailBound` at 532 lines and `Localisation` at 164 lines) were over-engineered relative to the actual proof needs. The eventual proof (per GPT's strategy memo) used a global first-order remainder bound and never invoked tail bounds at the strength developed. Claude's defaults pull toward more general infrastructure than is strictly needed. GPT-5.5 Pro's recommendations consistently pulled in the opposite direction: "don't build Watson machinery", "skip generic step 10/11", "specialise to $n = 0, 1, 2, 3$". Strategic minimalism was GPT's contribution; tactical maximalism was Claude's default.

No `sorry`, ever. The aks-style discipline held throughout. At every commit, `scripts/sorries` reported zero occurrences. Several intermediate states had stub theorems whose proofs were sketched in comments but the file was never committed with an unfilled `sorry`. This discipline made it possible to interrupt and resume work without the sense of a growing debt.

10 Cost and duration

The full session usage:

Wall-clock duration	5h 44m 26s
API duration	2h 30m 48s
Total cost	$198.53
Lines added (gross)	6050
Lines removed (gross)	1568
Lines retained (net)	∼3279 (final Lean) + ∼1200 (markdown, scripts)
Per-model usage:	
Claude (Opus 4.7, 1M context)	$198.14
input tokens	6.7k
output tokens	646.2k
cache read tokens	324.2m
cache write tokens	3.2m
Haiku 4.5 (auxiliary)	$0.39
input / output tokens	72.0k / 5.2k

A few features deserve comment.

The wall-clock vs API ratio. 5h 44m of wall-clock for 2h 30m of model time means roughly 56% of the wall-clock was spent waiting on the human (DM was reading screenshots, drafting messages, deciding what to do next, or doing other work) and 44% was spent waiting on the model. This is a useful sanity check: the bottleneck in this kind of session is the human, not the model.

Cache read vs input tokens. 324m cache-read tokens vs 6.7k fresh input tokens. Almost the entire conversational context was served from the prompt cache. Without prompt caching this session would have cost roughly an order of magnitude more.

Lines added vs lines retained. 6050 lines added, 1568 removed, leaving a final 3279 lines of Lean. So roughly 50% of what was written was kept verbatim, 25% was rewritten, and 25% was discarded outright. This is consistent with the impression from the inside: there were perhaps four or five points where a sub-goal had to be retried or restructured (the over-engineered `TailBound` stage, the bridge step, several `integral_add`/`integral_sub` pattern unification failures), and each retry cost a few hundred lines of churn.

Cost per Lean line. \$198.53 / 3279 retained lines = \$0.06 per kept line of Lean. As a back-of-envelope: at \$50–150/hr for human formalisation effort and a heuristic of 50–200 lines of Lean per human-hour for novel content, the equivalent human cost would be roughly \$800–\$10 000 for the same output. The advantage compounds with the human's wall-clock savings: a researcher who would have spent 30 hours on this in 2025 spent under 6 hours of wall-clock attention in 2026, almost all of it directional rather than tactical.

The Haiku rounding error. The 39 cents of Haiku 4.5 usage was auxiliary tooling (web fetches, project audits). It is a striking marker of how cheap small auxiliary models have become, and an argument for delegating routine searches and audits to them as a matter of course.

11 Lessons

1. **Borrow infrastructure aggressively.** The `aks`-derived audit script, `lean-search` wrapper, and `CLAUDE.md` structure saved at least a day of setup time. A project of this size cannot afford to grow its own tooling from scratch.

2. **Strategy consultations are cheap and load-bearing.** Three short GPT-5.5 Pro consultations changed the trajectory of the entire project. Without the first, we would have built Watson–Erdélyi infrastructure for weeks. Without the third, we would have stopped at the J_n-form theorem. The *cost* of a strategy consultation is a few minutes; the *value* is the difference between the project ending and the project not ending.

3. **The hypothesis is part of the theorem.** The discriminant condition $\alpha^2 < 3\lambda\gamma$ is not a regrettable technicality but a structural feature: without it, the primer's formula is false (easy counterexample: α very large with λ, γ small). The primer treats this as obvious; in Lean it becomes explicit. This is exactly the bookkeeping benefit DM wanted from formalisation in the first place.

4. **Specialise early.** GPT's recommendation to specialise to $n = 0, 1, 2, 3$ rather than build a generic parity framework was prescient. Generic frameworks pay off across many uses; for a single-target proof they are a tax.

5. **Avoid `Real.rpow` when integer powers suffice.** Mathlib's `Real.rpow` interacts poorly with several common tactics. For exponents that are known integers, use `HPow` directly; the bookkeeping savings compound.

6. **Notice thrashing early; escalate to a different model.** DM's intervention to consult 5.5 Pro on the bridge step was the right move at the right time. Three failed tactical attempts in a row is the canonical sign. The intervention was not "DM solved the problem" but "DM noticed and changed the configuration".

7. **Mathlib gap maps are project-specific.** About 80% of this proof was new content built on top of Mathlib's foundations, but the *shape* of what was missing was specific: pointwise Taylor bounds, Mill's-ratio family, global Laplace remainder. A different theorem in SLT would surface a different gap map. The library that this project contributes (the `Laplace` library) starts to fill in the analytic-asymptotics part of that map.

8. **The point was not the theorem; the point was the loop.** The covariance formula is not used directly anywhere in Timaeus's research roadmap. What is used is the loop: "write LaTeX, formalise the key claim in Lean, present both to a human reviewer who reads the Lean statement to verify the LaTeX claim". This project is one instance of that loop. Whether the loop scales depends on whether subsequent projects can be formalised at similar cost; the present project took roughly a morning of human time, 5h 44m of wall-clock, 2h 30m of API time, and $199 of model usage to produce 3279 lines of Lean (see §10). If that cost ratio holds, the loop is viable.

12 Possible next steps

Tag the primer with a Lean reference. The primer's equation (4.10) ($_t[x^2, x] = -2\alpha/(\lambda^3 t^2) + o(t^{-2})$) has been tagged with a `\leanref` macro pointing at the formalised theorem `cov_anharmonic_asymptotic`, pinned to commit `bfe350c`. This is the lightest-weight version of the loop described in the "Formalisation First Steps" note: a reader of the primer can click through to the Lean statement and verify in twenty minutes that what is stated in the LaTeX matches what is proved in Mathlib.

Adopt `leanblueprint` once a second result is formalised. `leanblueprint` is the de-facto tool for projects of this kind: it is what underpins the Liquid Tensor Experiment, the Polynomial Freiman–Ruzsa formalisation, and several other major Mathlib formalisation projects. It provides LaTeX macros (`\lean{name}`, `\leanok`, `\uses{...}`) that mark up paper claims with their corresponding Lean names and dependency edges, and it generates an interactive HTML dependency graph in which each node is hyperlinked to the Lean source with build/proof status. For a single formalised theorem this is overkill — the `\leanref` macro above does the job in one line — but as soon as a second primer result is formalised (e.g. `lem:laplace_cov` as the next natural target), the dependency graph becomes genuinely useful. The setup cost is roughly an afternoon of work, mostly configuring the blueprint scaffold and porting the existing `\leanref` call into the `\lean / \leanok / \uses` form.

Multivariate Laplace. The current formalisation is purely one-dimensional. The primer's main result (`lem:laplace_cov2` at the multivariate level) is the d-dimensional Wick-contraction formula ${}_t[\phi, \psi] = -\frac{1}{2t^2} T_{ijk}[\cdots] + o(t^{-2})$, which our `TailBound.lean` and `Localisation.lean` were over-engineered for in anticipation. The 1D case proved that the toolkit works; the multivariate case will exercise a different part of Mathlib (`Matrix` algebra, multivariate Gaussian, contraction with symmetric tensors).

Backport into Mathlib. Several pieces of the `Laplace` library are general enough to live in Mathlib proper: the two-sided $|e^{-z} - (1 - z)|$ scalar bound (`ScalarBound.lean`), the polynomial-times-Gaussian half-line tail bounds (`TailBound.lean`), and possibly the rescaling identity. A PR to Mathlib upstreaming these would shorten future formalisations and is the kind of contribution that pays off in goodwill across the Lean community.

13 Acknowledgements

- Geoffrey Irving's `aks` repository provided the model and several concrete artefacts (audit script, search wrapper, project layout); see §4.
- GPT-5.5 Pro provided three strategic consultations preserved verbatim at `gpt_responses/`.
- The mathematical content tracks the SLT Susceptibility Primer (Elliott–Murfet, 2026).
- The formalisation depends throughout on Mathlib.
- Edmund Lau and Geoffrey Irving for earlier conversations about formalisation strategy at Timaeus.